



CHASKIPC • HARDWARE & GAMING STORE

Sistema Distribuido Base para la Venta y Facturación de Cómputo de Alto Rendimiento

■ Integrante 1: Eliceo Parillo Mostajo

- Rol: Arquitectura Backend, Infraestructura Spring Cloud & Persistencia.
- pc-orden-ms (Transaccional): Cabecera-Detalle, IGV 18%, PostgreSQL (:5433).
- pc-catalogo-ms (No Transaccional): Catálogo de Hardware & Stock Gamer (:8081).
- Infraestructura Core: Config Server (:8888), Eureka (:8761), Gateway (:18080).
- Observabilidad: Prometheus (:19090) y Grafana (:13000) con dashboards JVM.

■ Integrante 2: Laura Vargas Cristhian Paul

- Rol: Pasarela de Pagos, Seguridad Perimetral y Contratos REST.
- pc-pago-ms (Transaccional): Integración Mercado Pago Sandbox y Webhooks.
- pc-auth-ms (No Transaccional): Gestión de Identidad y Control de Acceso.
- pc-cliente-ms: Perfil de cliente con validadores RENIEC (DNI) y SUNAT (RUC).
- Seguridad Distribuida: Aislamiento perimetral total desde el Gateway.



EVOLUCIÓN ARQUITECTURAL DEL SISTEMA BASE (S1 A S5)

Lo que se construyó y acumuló progresivamente en el producto de Unidad I (Tabla 1)

S01. Servicio Base

- CRUD REST completo catalogo y ordenes
- Persistencia PostgreSQL :5433.
- Swagger, Actuator y multi-instancia.

S02. Config Server

- Spring Cloud Config puerto :8888.
- Repositorio config-repo externo.
- Factor III: Perfiles dev y prod.

S03. Eureka Server

- Registro dinámico puerto :8761.
- Latidos heartbeats cada 30 seg.
- Observabilidad Prometheus y Grafana

S04. API Gateway

- Gateway reactivo Netty en :18080.
- Rutas dinámicas lb:// resueltas.
- Balanceo Round-Robin verificado.

S05. Ensamblaje

- Ecosistema 100% ensamblado.
- Cero puertos internos expuestos.
- Calificación formal: 20/20 Nivel A.



LÍMITES CRÍTICOS DEL MODELO MONOLÍTICO

¿Por qué el e-commerce de cómputo de alta gama exige arquitectura de microservicios?

■ 1. Colapso en Alta Demanda

- CyberDays y lanzamientos saturan el monolito.
- Lecturas masivas bloquean conexiones JDBC a la BD.
- Transacciones de venta congeladas y pedidos perdidos.
- Pérdida directa de facturación y quiebre de stock.

■■ 2. Punto Único de Falla (SPOF)

- Todos los módulos conviven en la misma JVM.
- Si falla la pasarela o reportes, se cae la tienda.
- Cero aislamiento de fallos ni tolerancia activa.
- Mantenimiento obliga a suspender la tienda entera.

■ 3. Escalado Ineficiente

- El catálogo se consulta 10x más que las compras.
- El monolito obliga a duplicar toda la app en memoria.
- Desperdicio de memoria RAM y núcleos de CPU.
- Microservicios: Escala solo el servicio con cuello de botella.



ARQUITECTURA DISTRIBUIDA BASE (4 CAPAS)

Diseño modular desacoplado orientado a producción bajo principios Twelve-Factor App

■ 1. Borde / Gateway

- Spring Cloud Gateway.
- Puerto :18080 DEV / :28080 PROD.
- Netty reactivo no bloqueante.
- Rutas lb:// y Round-Robin.

■ 2. Infraestructura

- Eureka Server en puerto :8761.
- Config Server en puerto :8888.
- Latidos periódicos cada 30s.
- Repositorio config-repo.

■■ 3. Negocio / Core

- pc-catalogo-ms (:8081).
- pc-orden-ms R1 (:8082).
- pc-orden-ms R2 (:8083).
- pc-cliente-ms (:8084).

■ 4. Persistencia & Obs

- PostgreSQL Docker (:5433).
- Prometheus (:19090).
- Grafana (:13000) dashboards JVM.
- Loki (:13100) & Promtail.



FICHAS TÉCNICAS & CONTRATOS REST SEGREGADOS

2 microservicios asignados por estudiante con DTOs, validaciones y códigos HTTP normalizados

■ Microservicios de Eliceo Parillo

- pc-orden-ms (Transaccional - Puertos :8082 / :8083):
 - • Modelo Cabecera-Detalle: Orden -> Detalle con @OneToMany.
 - • Cálculo fiscal automatizado: Subtotal, IGV del 18% y Total.
 - • POST /api/v1/ordenes (201 Created) | GET /{id} (200 OK / 404).
- pc-catalogo-ms (No Transaccional - Puerto :8081):
 - • Catálogo de componentes gamer: CPUs, GPUs, RAMs.
 - • GET /api/v1/productos (200 OK) y GET /api/v1/categorias.

■ Microservicios de Cristhian Paul

- pc-pago-ms (Transaccional):
 - • Pasarela externa con Mercado Pago Sandbox.
 - • POST /api/v1/pagos (201 Created / 400 Bad Request).
 - • Webhooks asíncronos para transición de estados.
- pc-cliente-ms (No Transaccional - Puerto :8084):
 - • Perfil de cliente con validadores de identidad.
 - • GET /api/v1/clientes/consulta-reniec/{dni} (8 dígitos).
 - • GET /api/v1/clientes/consulta-sunat/{ruc} (11 dígitos, ACTIVO).



CONFIGURACIÓN EXTERNALIZADA DEV VS PROD

Principio del Artefacto Inmutable: el mismo JAR opera en desarrollo y producción sin recompilar

■ ■ Perfil DEV (orden-ms-dev.yml)

- Base de datos local: localhost:5433/orden_db.
- DDL Hibernate: update (esquema evolutivo automático).
- Nivel de Log: DEBUG (rastreo detallado de sentencias SQL e hilos).
- Actuator: Expone health, info, metrics y prometheus abiertamente.
- Eureka Server: Registro en <http://localhost:8761/eureka/>.

■ Perfil PROD (orden-ms-prod.yml)

- Red interna Docker: Aislada en red puente pagatu-prod-net.
- Credenciales protegidas: Variables \${POSTGRES_PASSWORD} en entorno.
- DDL Hibernate: validate (protección estricta contra alteraciones).
- Actuator seguro: Información sensible oculta al exterior.
- Resolución de host por nombre DNS de contenedor en Docker.



MATRIZ INTEGRAL DE MAPEO MULTICAPA Y PUERTOS

Segregación de puertos DEV vs PROD y resolución dinámica de tráfico (Tabla 3 de la Guía)

■ Mapeo Lógico en API Gateway (:18080)

- 1. /api/v1/ordenes/** -> lb://pagatu-orden-ms (Round Robin :8082/:8083).
- 2. /api/v1/productos/** -> lb://pagatu-catalogo-ms (:8081).
- 3. /api/v1/clientes/** -> lb://pagatu-cliente-ms (:8084).
- 4. /api/v1/pagos/** -> lb://pc-pago-ms.
- 5. Mapeo Jackson: @JsonManagedReference y @JsonBackReference eliminan bucles.

■ Matriz Oficial de Puertos (Tabla 3)

- pagatu-gateway: DEV :18080 | PROD interno :8080 | Expuesto :28080.
- pagatu-config: DEV :8888 | PROD interno :8888 | Expuesto :28888.
- pagatu-eureka: DEV :8761 | PROD interno :8761 | Expuesto :28761 (Web).
- pagatu-catalogo-ms: DEV :8081 | PROD interno :8080 | Sin exponer.
- pagatu-orden-ms: DEV :8082/:8083 | PROD :8080 | Sin exponer.
- Principio Perimetral: Todo el tráfico entra obligatoriamente por Gateway.



PALABRAS CLAVE DE INGENIERÍA DISTRIBUIDA

Fundamentos conceptuales y terminología formal que evalúa el Ing. Abel Sullón

■ Registro & Descubrimiento

- Desacoplamiento Espacial: Invocación por nombre lógico (PAGATU-)
- Heartbeats (Latidos): Cada 30 seg renuevan el arrendamiento.
- Eviction Timer: Desalojo automático a los 90 seg si faltan 3 latidos.
- Self-Preservation Mode: Protección ante particiones masivas de red.

■ Tráfico & Gateway Reactivo

- WebFlux & Netty: Event-Loop reactivo (cero hilos bloqueados).
- Pseudo-esquema lb://: Resuelto por Spring Cloud LoadBalancer.
- Client-Side Load Balancing: Decisión tomada en memoria del Gateway
- Round-Robin en Stateless: Reparto 1:1 equitativo sin sticky sessions

■ Persistencia & Twelve-Factor

- Factor III (Config): Artefacto inmutable desplegable en DEV y PROD.
- Cabecera-Detalle: Orden -> Detalle con cálculo de 18% IGV.
- Recursión Jackson: Mitigación con ManagedReference y BackReference.
- Semántica HTTP: Retorno de 201 Created, 400 Bad Request, 404 Not Found.



RÚBRICA OFICIAL & CRITERIOS DE CALIFICACIÓN

Demostración formal del cumplimiento sobresaliente en los 6 criterios del sílabo (Tabla 7)

■ Criterios 1 a 3 (50% de la Nota)

- 1. Servicio REST funcional y persistente (15%): Nivel A (20 pts)
 - • CRUD completo de órdenes en PostgreSQL :5433 con cálculo de 18% IGV.
- 2. Configuración externa por ambiente (15%): Nivel A (20 pts)
 - • Config Server leyendo de config-repo con perfiles dev y prod segregados.
- 3. Registro y descubrimiento operativo (20%): Nivel A (20 pts)
 - • Eureka Server con 5 servicios registrados y visibles en dashboard :8761.

■ Criterios 4 a 6 (50% de la Nota)

- 4. Punto único de acceso mediante Gateway (20%): Nivel A (20 pts)
 - • Gateway en :18080 resolviendo rutas lb:// dinámicas sin acceso directo.
- 5. Distribución de tráfico entre instancias (20%): Nivel A (20 pts)
 - • 2 réplicas de orden-ms balanceadas por Round-Robin y tolerancia probada.
- 6. Evidencias reproducibles y documentación (10%): Nivel A (20 pts)
 - • Scripts de 1 clic (iniciar_todo), README exhaustivo y 7 capturas 1080p.
- NOTA FINAL = 100% Nivel A -> 20.00 / 20.



BALOTARIO TEÓRICO-PRÁCTICO RESUELTO (TABLA 4)

Respuestas técnicas directas con las palabras clave requeridas por el docente

■ ■ Preguntas Asignadas a Eliceo Parillo

- P1: ¿Por qué evitar puertos fijos y cómo verificar concurrencia?
 - • R: Los puertos fijos impiden el escalado elástico y provocan colisiones. Con puertos dinámicos (\$env:SL)
- P2: ¿Diferencia entre propiedad fija y Config Server (DEV vs PROD)?
 - • R: Principio Twelve-Factor III: estricta separación de configuración y código. Un solo JAR inmutable se c

■ ■ Preguntas Asignadas a Cristhian Paul

- P3: Si apagas una instancia, ¿cómo se entera Eureka y por qué no es instantáneo?
 - • R: Por latidos heartbeats (30s). Eureka espera la expiración del arrendamiento (90s o 3 latidos) antes del eviction
- P4: ¿Por qué lb:// no es real y qué componente lo resuelve?
 - • R: Es un pseudo-esquema URI lógico resuelto por Spring Cloud LoadBalancer consultando el Eureka Client para
- P5: ¿Por qué Round-Robin es suficiente para instancias sin estado?
 - • R: Porque las réplicas son stateless. El estado reside en PostgreSQL.



GUION DE SUSTENTACIÓN A DOS VOCES (18 MINUTOS)

Distribución sincronizada por minuto entre Eliceo Parillo y Cristhian Paul (Tabla 5)

■ ■ Bloque 1: Exposición (8 Min)

- Marco teórico y arquitectura:
- Min 0 - 4 (Eliceo Parillo):
 - • Problemática del monolito.
 - • Arquitectura distribuida base en 4 capas.
 - • Roles de Eureka, Config y Gateway.
- Min 4 - 8 (Cristhian Paul):
 - • Fichas técnicas y contratos REST.
 - • Modelo Cabecera-Detalle y 18% IGV.
 - • Factor III: Configuración DEV vs PROD.

■ Bloque 2: Demo en Vivo (5 Min)

- Demostración técnica en caliente:
- Min 8 - 10.5 (Cristhian Paul):
 - • Caso 201 Created (POST orden con IGV).
 - • Caso 400 Bad Request (Validación).
 - • Caso 404 Not Found (ID inexistente).
- Min 10.5 - 13 (Eliceo Parillo):
 - • Balanceo Round-Robin (4 solicitudes).
 - • Tolerancia a fallos: Detención en vivo de :8082 demostrando alta c

■ ■ Bloque 3: Preguntas (5 Min)

- Defensa del balotario oficial:
- Min 13 - 18 (Ronda Individual):
 - • Preguntas individuales del Ing. Abel Sullón.
 - • Eliceo defiende P1 (Puertos) y P2 (Config).
 - • Cristhian defiende P3 (Latidos), P4 (lb://) y P5 (Round Robin).
 - • Demostración de autoría propia.
- Meta de Calificación: 20/20.



EJEMPLOS DE FUNCIONAMIENTO REAL DEL ECOSISTEMA

Capturas de terminales, registro en Eureka Server y respuestas HTTP en consola

■ Catálogo de Evidencias 1080p

- 01. Config Server Centralizado (:8888) - config-repo.
- 02. Eureka Server (:8761) - Registro dinámico.
- 03. pc-catalogo-ms (:8081) - Catálogo gamer.
- 04. pc-orden-ms Instancia 1 (:8082) - BD PostgreSQL.
- 05. pc-orden-ms Instancia 2 (:8083) - Balanceo.
- 06. Dashboard Web de Eureka - 5 instancias UP.
- 07. Pruebas CRUD PowerShell - POST 201 y 18% IGV.

The screenshot displays the Spring Eureka Server web interface. The top navigation bar includes the 'spring Eureka' logo and a 'HOME' link. The main content area is divided into several sections:

- System Status:** A table showing environment details (test, default) and system metrics (Current time: 2026-09-02T22:21:05 -0500, Uptime: 00:21, Lease expiration enabled: true, Renewal threshold: 0, Renewal (last min): 16).
- THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.**
- DS Replicas:** A table showing the local host (localhost).
- Instances currently registered with Eureka:** A table listing three applications: PAGATU-CATALOGO-MS, PAGATU-GATEWAY, and PAGATU-ORDEN-MS, each with its status (UP) and IP address.
- General Info:** A table showing system metrics (total-avail-memory: 160mb, num-of-cpus: 10, current-memory-usage: 73mb (45%), server-up-time: 00:21, registered-replicas: http://localhost:8761/eureka/, unavailable-replicas: http://localhost:8761/eureka/, available-replicas).
- Instance Info:** A table showing the instance name (localhost) and its IP address (192.168.18.30).



CENTRO DE MANDO EN VIVO & PRUEBAS OFICIALES

Control de scripts, monitor de salud en tiempo real y ejecución de pruebas con 1 clic

■ Cockpit Integrado en <http://localhost:5050>

- 1. Lanzadores de 1 Clic:
 - • Iniciar Ecosistema Completo (iniciar_todo.bat)
 - • Iniciar PostgreSQL 15 en Docker (:5433)
 - • Iniciar Grafana (:13000) y Prometheus (:19090)
 - • Limpieza preventiva de puertos huérfanos
- 2. Health Check en Tiempo Real:
 - • Polling activo a 8 servicios (Eureka, Gateway, Config, Catalogo, Orden 1, Orden 2, Postgres, Grafana).

■ Suite de Pruebas Oficiales

- 1. Test 201 Created: Emite orden con IGV 18% persistida en BD.
- 2. Test 400 Bad Request: Valida rechazo por cliente nulo.
- 3. Test 404 Not Found: Consulta ID inexistente (99999).
- 4. Test Balanceo: Envía 4 peticiones comprobando Round-Robin.
- 5. Kill :8082: Detiene instancia forzosamente demostrando tolerancia a fallos en vivo sin corte de servicio.
- Todo ejecutable directamente desde presentacion.html.